MEMORY FUNCTION

Memoization in Dynamic Programming



Solve → Store → Reuse

B.Tech • Design & Analysis of Algorithms

What is a Memory Function?

A memory function stores solutions of previously solved subproblems.

Definition

In Dynamic Programming, a memory function is a table/cache that stores the result of a subproblem after it is computed.

It is commonly associated with the top-down DP technique called memoization.

Recursive Problem + Cache = Memoization

Core Idea

If a state has already been solved → return its stored value.

Otherwise

Compute it → store it → return the result.

Memoization: The Top-Down DP Approach

Start with the original problem and recursively solve only the required states.

1. Check Cache

Before computing f(state), check whether its answer is already stored.

2. Compute

If not stored, recursively solve smaller subproblems using the recurrence.

3. Store + Return

Save the computed value in the memory table and return it to the caller.

Why Do We Need a Memory Function?

Naive recursion may solve the same state many times.

Without Memoization

- Repeated recursive calls
- Same subproblem recomputed
- Exponential growth for many recurrences
- Large amount of redundant work

With Memoization

- Each state is computed once
- Later calls read from cache
- Eliminates repeated work
- Complexity becomes dependent on the number of states

Example: Fibonacci Recursion

Fibonacci clearly demonstrates overlapping subproblems.

$$F(n) = F(n-1) + F(n-2)$$

Naive Recursion

F(5) calls F(4) and F(3).
F(4) again calls F(3).
F(3) is therefore computed repeatedly.
This creates overlapping subproblems.

Memoized Recursion

Compute F(3) once.
Store it in memo[3].
Every later request for F(3) returns memo[3].

Fibonacci: Repeated States vs Stored States

The memory function changes the amount of work.

State	Naive recursion	Memoized recursion
F(0)	Repeated in branches	Computed once
F(1)	Repeated in branches	Computed once
F(2)	Repeated	Computed once
F(3)	Repeated	Computed once
F(4)	Repeated	Computed once
F(5)	Result	Result from stored states

Main Observation

There are only O(n) distinct Fibonacci states: 0,1,…,n.

Result

Memoization reduces Fibonacci time from exponential to O(n).

How the Memory Function Works

A cache entry has two possible states: unknown or already computed.

Initially

memo[0..n] = UNKNOWN

No subproblem has been solved yet.

During Recursion

When f(x) is requested:

1. Check memo[x]
2. If known → return
3. Else compute f(x)

After Computation

memo[x] = answer

The same state can now be reused in O(1) lookup time.

Memoization Trace for F(5)

Only the first computation of each state performs real work.

Call	Action	Stored Result
F(5)	Compute	memo[5]=5
F(4)	Compute	memo[4]=3
F(3)	Compute	memo[3]=2
F(2)	Compute	memo[2]=1
F(1)	Base case	memo[1]=1
F(0)	Base case	memo[0]=0
Later F(3)	Cache hit	Return memo[3]

Cache Hit

A cache hit means the state has already been computed.

Key Benefit

No second recursive expansion is required for that state.

Memory Function Design Pattern

A general template for top-down dynamic programming.

```
MEMOIZED(state):

1. If state is already stored:
       return memo[state]

2. If state is a base case:
       answer = base value

3. Otherwise:
       answer = recurrence using
                smaller states

4. Store:
       memo[state] = answer

5. Return answer
```

Essential Components

- State definition
- Base cases
- Recurrence
- Cache / memory table

Correctness

Every cached value must correspond to exactly one well-defined DP state.

Pseudocode for Memoization

Generic top-down dynamic programming pattern.

```
MEMO-SOLVE(state):

if memo[state] is already computed:
    return memo[state]

if state is a base case:
    memo[state] = base_value
    return memo[state]

answer = combine(
    MEMO-SOLVE(smaller_state_1),
    MEMO-SOLVE(smaller_state_2),
    ...
)

memo[state] = answer
return memo[state]
```

Time

Usually proportional to the number of distinct states × transition cost.

Space

Cache space + recursion stack.

For Fibonacci: O(n) total auxiliary space.

C Program: Fibonacci Using Memoization

Simple implementation suitable for B.Tech DAA classes.

```c
#include <stdio.h>

#define MAX 100

long long memo[MAX];

long long fib(int n) {
    if (n == 0 || n == 1)
        return n;

    if (memo[n] != -1)
        return memo[n];

    memo[n] = fib(n - 1) + fib(n - 2);
    return memo[n];
}

int main() {
    for (int i = 0; i < MAX; i++)
        memo[i] = -1;

    printf("F(10) = %lld", fib(10));
    return 0;
}
```

Memory Function for Binomial Coefficient

Memoization can also be applied to C(n,k).

$$C(n,k) = C(n-1,k-1) + C(n-1,k)$$

Base Cases

C(n,0)=1

C(n,n)=1

If k<0 or k>n → 0

Memory Function

Before computing C(n,k), check memo[n][k].

If stored, return it immediately.

$$memo[n][k] \leftarrow \text{computed answer}$$

Worked Example: C(5,2) with Memoization

The same recurrence is evaluated top-down.

Start

Request C(5,2).

Not in memory → compute C(4,1) + C(4,2).

Recursive States

Compute smaller states.

Store each result.

If a state appears again, read it from memo.

Final

memo[5][2] = 10

Therefore C(5,2)=10.

C Program: Binomial Coefficient with Memoization

Top-down implementation.

```c
#include <stdio.h>

int memo[50][50];

int C(int n, int k) {
    if (k == 0 || k == n)
        return 1;

    if (k < 0 || k > n)
        return 0;

    if (memo[n][k] != -1)
        return memo[n][k];

    memo[n][k] =
        C(n-1, k-1) + C(n-1, k);

    return memo[n][k];
}

int main() {
    /* initialize memo[][] to -1 */
    printf("%d", C(6,3));  // 20
    return 0;
}
```

Advantages and Limitations of Memoization

A practical comparison for algorithm design.

Advantages

- Avoids repeated subproblem computation
- Natural extension of recursive solutions
- Computes only required states
- Easy to derive from a recurrence
- Useful when state space is large but sparsely visited

Limitations

- Recursion adds call-stack overhead
- Deep recursion may cause stack overflow
- Cache consumes memory
- Slightly harder to debug than iterative DP
- All states may still be visited in worst cases

Complexity Analysis

Memoization complexity depends on the number of distinct states.

Naive Fibonacci

Time ≈ O(2^n)

Space = O(n) recursion stack.

Memoized Fibonacci

Time = O(n)

Space = O(n) cache + stack.

Binomial Memoization

O(nk) distinct states

O(nk) cache space.

DP principle: #states × work per state

Important

Memoization improves time by eliminating recomputation, but it does not eliminate the need to store results.

When Should You Prefer Memoization?

Use it when the problem naturally fits a top-down formulation.

Good Situations

- Recurrence is easy to express recursively
- Only a subset of states may be needed
- State space is large
- You want to convert recursion into DP with minimal structural change
- Problem has overlapping subproblems

Be Careful When

- Recursion depth may be very large
- Iterative traversal is simpler
- Memory is limited
- Almost every state will be visited anyway
- Stack overhead matters

Numerical Practice Questions

B.Tech / DAA exam-oriented practice.

Q1

Using memoization, calculate F(6). List memo[0] through memo[6].

Answer: F(6)=8

Q2

How many distinct states are computed for F(8) with memoization?

Answer: 9 states

Q3

Calculate C(5,2) using the memoized recurrence.

Answer: 10

Q4

Calculate C(6,3) using memoization without the factorial formula.

Answer: 20